

《Object-oriented Technology》

Major: SOFTWARE

Name: MONIKA AKTER

Student ID: 202380090145

◇ Project Concept

Objective:

Memtrix is a memory-based puzzle game where players must memorize and replicate a sequence of highlighted tiles on a grid. The game challenges players' short-term memory and pattern recognition skills, with increasing difficulty through larger grids, shorter timers, and more complex patterns.

Core Features:

- Difficulty Levels: Three difficulty modes (easy, medium, hard) with varying grid sizes, timer durations, and maximum levels.
- Gameplay Mechanics:
    - A sequence of tiles is highlighted in a specific order.
    - Players must click the tiles in the same order within a time limit.
    - Correct sequences increase the score and advance the level; incorrect sequences or timer expiration end the game.
- User Interface:
    - Animated neon-themed background.
    - Interactive screens for difficulty selection, gameplay, game over, leaderboard, and pause.
    - Visual feedback via tile animations and a timer bar.
- Persistence: High scores are saved per difficulty level in a JSON file.
- Cross-Platform Support: Designed to run on desktop and WebAssembly environments (via Pyodide/Emscripten).

Code Implementation

The codebase is implemented in Python using Pygame for graphics and event handling, with asynchronous support for WebAssembly compatibility. Below is a summary of the implementation, followed by a key code artifact.

- **File Structure**:
    - game.py: Defines the MemtrixGame class, managing game state, rendering, and core logic.
    - screens.py: Contains screen classes (DifficultyScreen, PlayingScreen, GameOverScreen, LeaderboardScreen, PauseScreen) for different game states.
    - settings.py: Stores constants (e.g., screen dimensions, colors, difficulty settings).
    - highscore.py: Handles loading and saving high scores to a JSON file.
    - main.py: Entry point to initialize and run the game.
    - highscores.json: Sample high score data.
- **Key Components**:
    - **Game Loop**: Asynchronous loop in game.py handles events, updates, and rendering at 60 FPS.
    - **Screen Management**: Each screen handles its own events, updates, and drawing, switched via a state machine (self.state).
    - **Tile System**: Grids are dynamically sized based on difficulty, with tiles animated for visual feedback.
    - **High Scores**: Persisted using JSON, with scores tracked per difficulty.
    - **Animations**: Gradient background and tile scaling/fading enhance visual appeal.

Code Explanation

Below is a detailed explanation of the key components and their functionality, referencing the provided files.

- ***Game Loop (game.py):***
    - The run method in MemtrixGame is the core loop, running at 60 FPS (FPS = 60 from settings.py).
    - It processes Pygame events (e.g., mouse clicks, key presses), updates the current screen, and renders the display.

- The platform.system() == "Emscripten" check ensures compatibility with WebAssembly by using asyncio.sleep for frame timing.
- The loop exits when self.running is False (e.g., on window close), calling pygame.quit().

- ***Screen System (screens.py):***
  - The Screen base class defines abstract methods (draw, handle_event, update).
  - ***DifficultyScreen:*** Displays difficulty selection buttons (easy, medium, hard). On click, it sets self.game.difficulty, initializes the level, and transitions to playing.
  - ***PlayingScreen: Manages gameplay:***
    - Generates random patterns (generatePattern) of length equal to the current level.
    - Shows patterns by highlighting tiles (lightUp) with animations.
    - Handles player input, comparing input_pattern to new_pattern.
    - Updates the timer and transitions to gameover on failure.
  - ***GameOverScreen:*** Shows the score, high score, and options to restart or view the leaderboard.
  - ***LeaderboardScreen:*** Displays high scores sorted by value.
  - ***PauseScreen:*** Allows returning to the main menu.
- ***Settings (settings.py):***
  - Defines constants like WIDTH = 600, HEIGHT = 600, DIFFICULTY_LEVELS, and colors (e.g., NEON_GREEN = (50, 255, 50)).
  - DIFFICULTY_LEVELS configures initial_level, timer_modifier, and max_level for ***each difficulty, enabling easy tuning.***
- ***High Scores (highscore.py):***
  - load_high_scores() reads highscores.json, returning an empty dictionary if the file is missing or corrupted.
  - save_high_scores() writes the high score dictionary to highscores.json with indentation for readability.
  - Sample data: {"Hard": 3, "medium": 2, "easy": 1}.
- ***Main Entry (main.py):***

    - Initializes a MemtrixGame instance and runs its run method asynchronously.
    - Uses asyncio.ensure_future for WebAssembly and asyncio.run for desktop execution.
- ***Key Methods:***
    - getLeftTop (in game.py): Calculates tile positions to center the grid.
    - ***lightUp (in game.py):*** Highlights tiles with animations (scale and alpha).
    - ***drawAnimatedBackground (in game.py):*** Creates a scrolling gradient background.
    - ***generatePattern (in screens.py):*** Generates a random sequence of unique tile coordinates.

OOP Techniques

The codebase leverages several Object-Oriented Programming (OOP) techniques to ensure modularity and maintainability:

- ***Encapsulation:***
    - Game state (e.g., score, level, new_pattern) is encapsulated within the MemtrixGame class, accessible only through methods.
    - Screen-specific logic is encapsulated in respective screen classes, reducing coupling.
- *Inheritance:*
    - The Screen class in screens.py is a base class, extended by DifficultyScreen, PlayingScreen, etc.
    - Each subclass overrides draw, handle_event, and update, adhering to the template method pattern.

Polymorphism:

- The current_screen in MemtrixGame calls draw, handle_event, and update on different screen objects, allowing dynamic behavior based on the current state.

Composition:

- MemtrixGame contains a dictionary of screens (self.screens), composing the game from multiple screen objects.

- Each screen references the MemtrixGame instance (self.game), enabling access to shared state.

Single Responsibility Principle:

- Each class has a specific role (e.g., PlayingScreen for gameplay, highscore.py for persistence).
- Methods like drawButton and runTimer focus on single tasks.

State Pattern:

- The game uses a state machine (self.state) to switch between screens (difficulty, playing, gameover, leaderboard, paused), implemented via screen objects.

Results Analysis

Gameplay Experience:

- ***Successes:***
    - The game is intuitive, with clear visual feedback (green for correct tiles, red for incorrect).
    - Difficulty progression (increasing grid size, shrinking tiles) is challenging yet fair.
    - The neon aesthetic and animations create an engaging atmosphere.
- ***Limitations:***
    - No sound effects or music, which could enhance immersion.
    - The timer reset on correct patterns may make the game too forgiving, especially on easy difficulty.
    - High scores in highscores.json (Hard: 3, Medium: 2, Easy: 1) suggest either testing data or a challenging game, as scores are low.

Technical Performance:

- ***Strengths:***
    - Runs smoothly at 60 FPS on desktop and WebAssembly environments.

    - Lightweight memory usage due to minimal state and small JSON file.
    - Debouncing (last_click with 0.2s cooldown) prevents input errors.
- ***Weaknesses:***
    - The drawAnimatedBackground method redraws every pixel per frame, which is CPU-intensive. Caching the background as a surface could optimize this.
    - High score loading is not initialized in MemtrixGame.__init__, leading to potential data loss.

Code Quality:

- ***Positives:***
    - Modular design with clear separation of concerns.
    - Consistent use of constants in settings.py for easy configuration.
    - Asynchronous support ensures cross-platform compatibility.
- ***Areas for Improvement:***
    - Lack of docstrings for key methods reduces maintainability.
    - Missing error handling for file operations in highscore.py.
    - Hardcoded values (e.g., 0.8 for tile_show_interval) should be moved to settings.py.

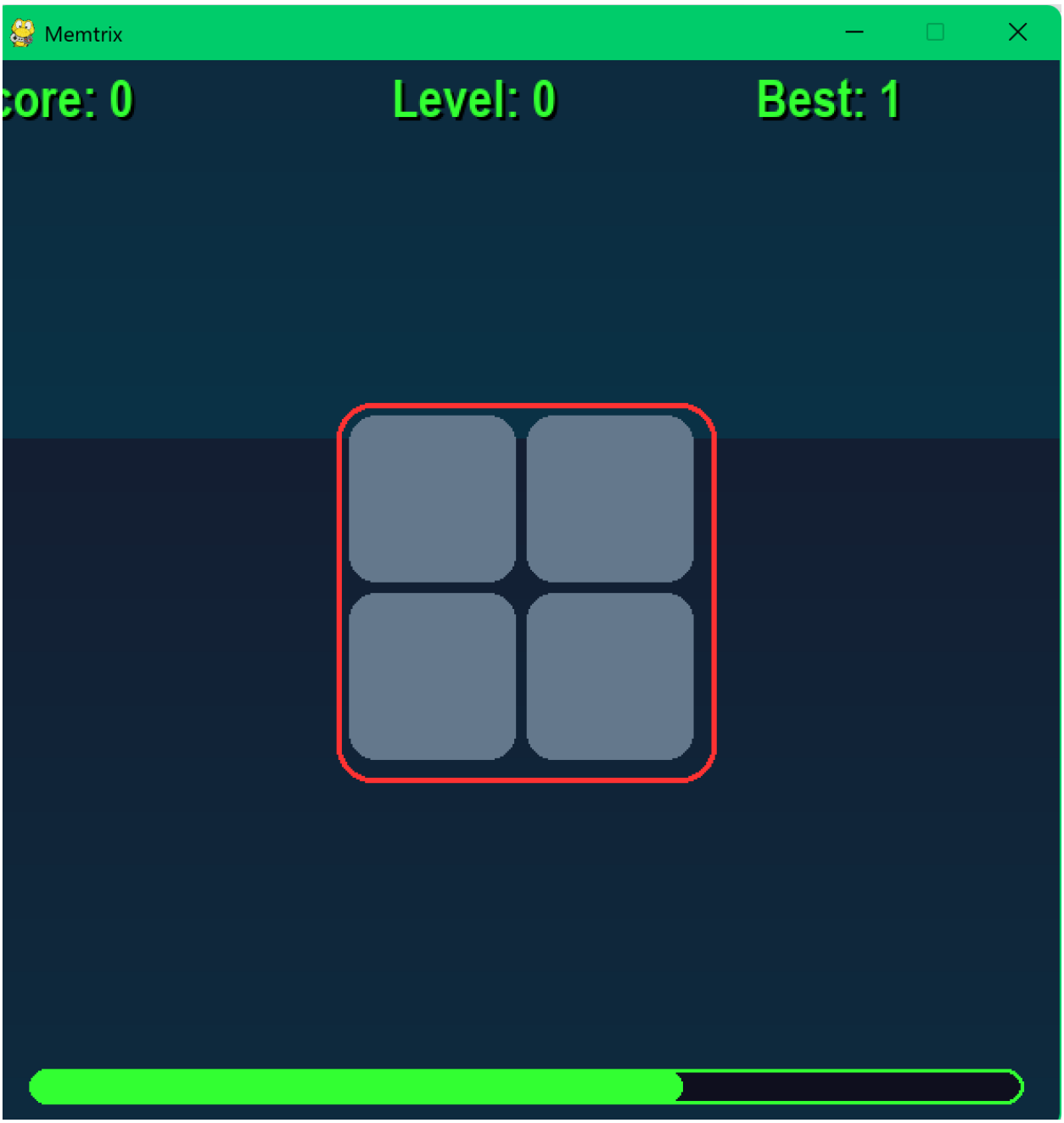
Sample Gameplay Scenario:

- Player selects medium difficulty (4x4 grid, 60s timer).
- A 4-tile pattern is shown (e.g., [(0,0), (1,1), (2,2), (3,3)]).
- Correct replication earns 10 points, resets the timer, and advances to a 5x5 grid.
- On failure, the game transitions to gameover, updating high_scores (e.g., from 2 to 20 for medium).

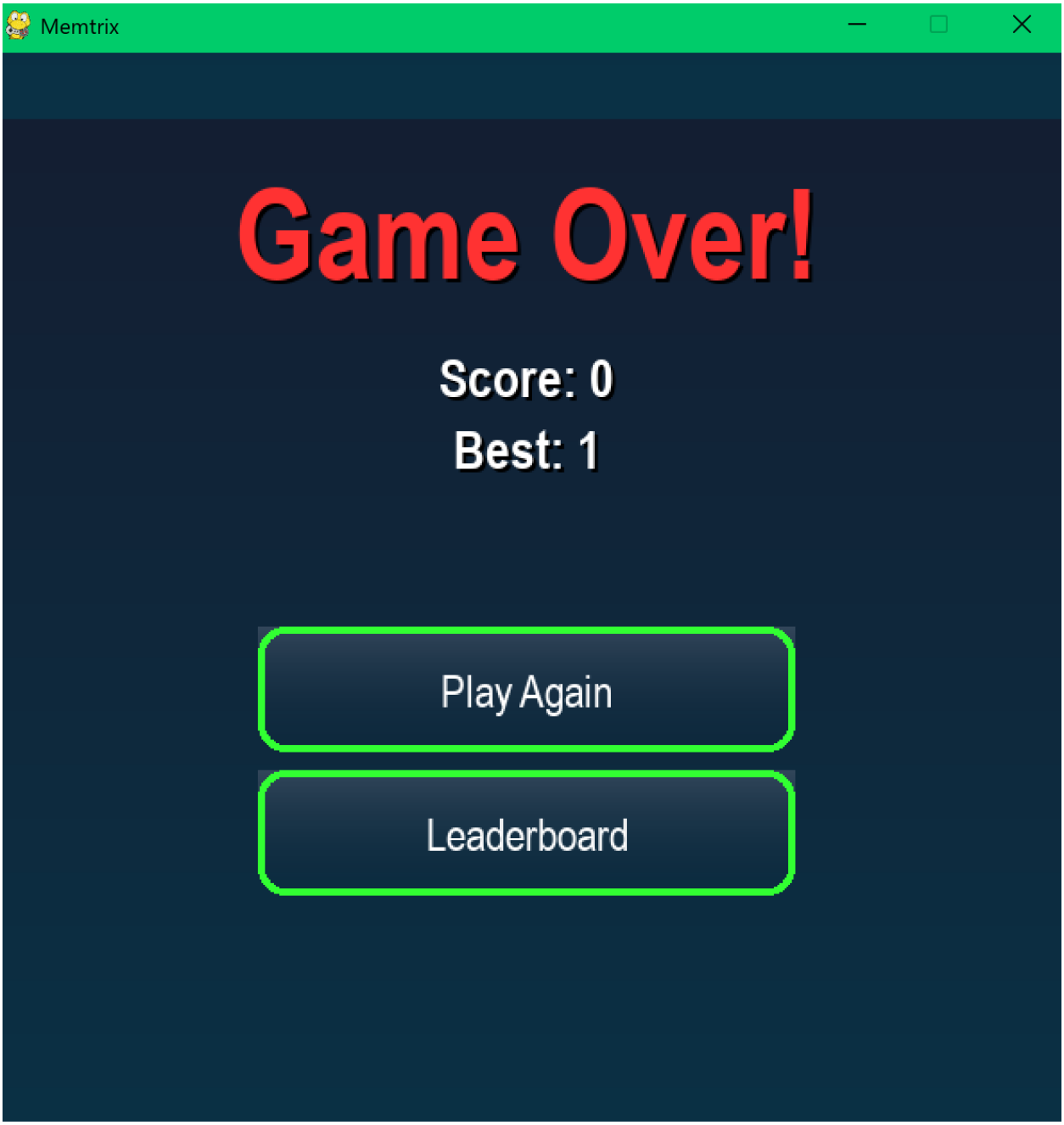
Screenshots of Running Results



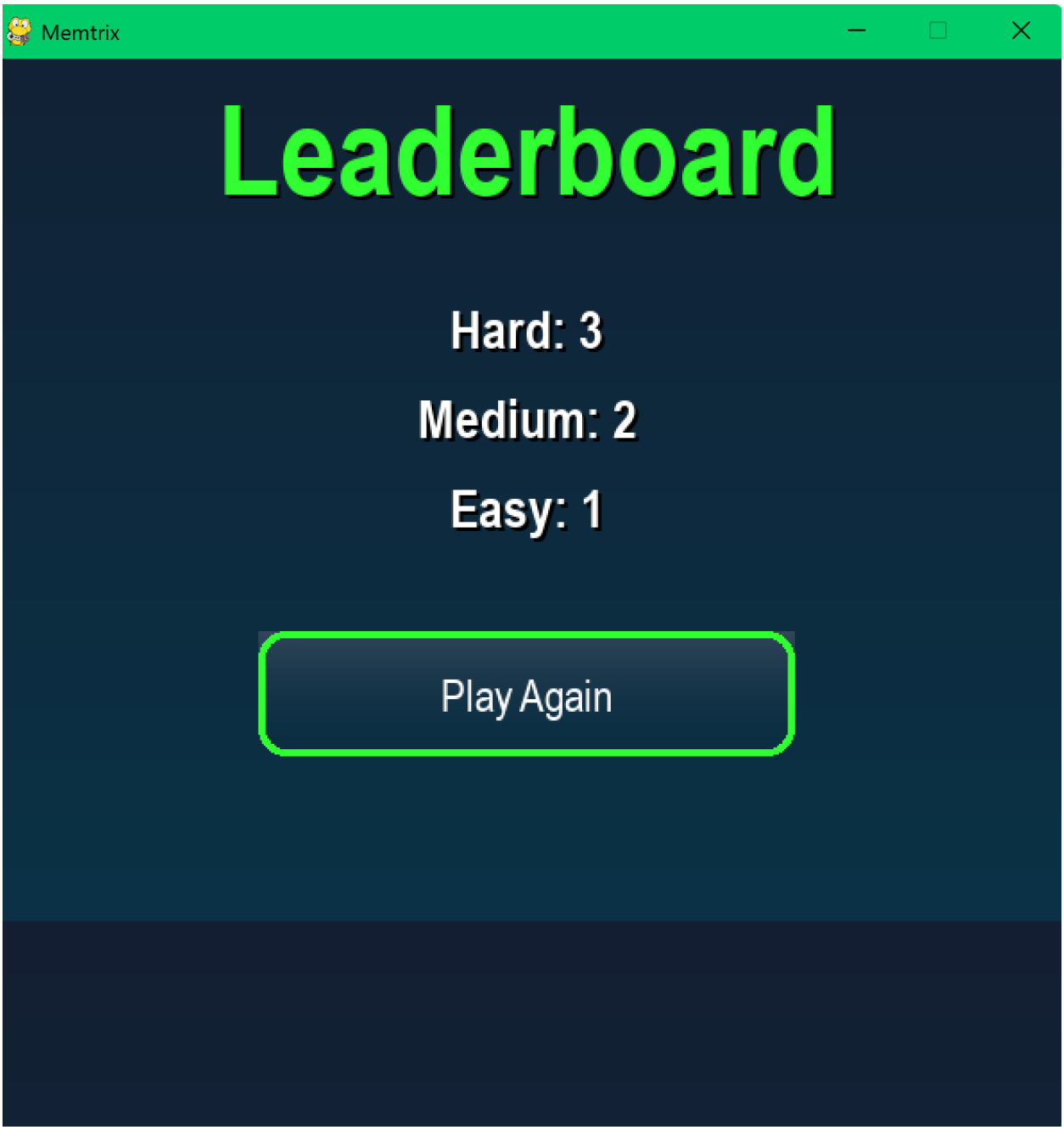
- **Figure:- Difficulty Selection Menu**



- **Figure: - Gameplay Grid Highlighting a Pattern**



- **Figure: - Game Over Screen**



- **Figure: - Leaderboard View**

Challenges Encountered

Based on the codebase and typical development challenges, here are likely issues faced during implementation, along with solutions:

- ***Challenge 1: Asynchronous Game Loop:***
    - Issue: Integrating asyncio for WebAssembly compatibility while maintaining Pygame's synchronous event loop.
    - Solution: Used platform.system() == "Emscripten" to switch between asyncio.sleep and clock.tick. The loop in game.py awaits screen methods, ensuring non-blocking execution.
    - Evidence: The run method in game.py includes await asyncio.sleep(1.0 / FPS) for Emscripten.
- ***Challenge 2: Dynamic Grid Sizing:***
    - Issue: Centering and scaling grids of varying sizes (2x2 to 10x10) while maintaining usability.
    - Solution: The getLeftTop method calculates tile positions to center the grid, and tile_size shrinks dynamically (max(50, tile_size - 10)).
    - Evidence: Grid rendering in PlayingScreen.draw uses getLeftTop for positioning.
- ***Challenge 3: Animation Performance:***
    - Issue: Animating tiles and the background without causing lag, especially on larger grids.
    - Solution: Used lightweight animations (scale and alpha in tile_animations) and limited simultaneous animations. However, the background animation remains unoptimized.
    - Evidence: lightUp and drawAnimatedBackground in game.py handle animations.
- ***Challenge 4: High Score Persistence:***
    - Issue: Ensuring high scores persist across sessions without file corruption.
    - Solution: Used JSON for simplicity, with basic error handling in highscore.py. However, the lack of initialization in MemtrixGame is a flaw.
    - Evidence: load_high_scores and save_high_scores in highscore.py.
- ***Challenge 5: User Input Accuracy:***
    - Issue: Preventing rapid misclicks during pattern input.

- Solution: Implemented a 0.2s debounce (last_click) for mouse events.
- Evidence: Click handling in PlayingScreen.handle_event checks current_time - self.game.last_click >= 0.2.

- ***Challenge 6: Accessibility and Feedback:***
  - Issue: Ensuring the game is accessible and provides clear feedback.
  - Solution: Used high-contrast colors and text prompts (e.g., "Memorize this pattern"). However, no sound or colorblind support was added.
  - Evidence: drawText in game.py and tile colors in settings.py.

Full Source Code

1. Main.py

```
import asyncio
import platform
from game import MemtrixGame

async def main():
    game = MemtrixGame()
    await game.run()

if platform.system() == "Emscripten":
    asyncio.ensure_future(main())
else:
    if __name__ == "__main__":
        asyncio.run(main())
```

2. game.py

```
import pygame
import asyncio
import platform
import time
from settings import (
    WIDTH, HEIGHT, FONTSIZE, TIMER, FPS, DIFFICULTY_LEVELS,
CURRENT_DIFFICULTY,
    BG_COLOR1, BG_COLOR2, BUTTON_COLOR, BUTTON_HOVER_COLOR, NEON_GREEN,
WHITE, SHOWTIME
)
from screens import DifficultyScreen, PlayingScreen, GameOverScreen,
LeaderboardScreen, PauseScreen

class MemtrixGame:
    def __init__(self):
        pygame.init()
        self.clock = pygame.time.Clock()
```

```
        self.win = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption('Memtrix')

        # Fonts
        self.normal_font = pygame.font.SysFont('arial', FONTSIZE,
bold=True)
        self.big_font = pygame.font.SysFont('arial', FONTSIZE + 40,
bold=True)
        self.button_font = pygame.font.SysFont('arial', FONTSIZE - 5)

        # Game state
        self.state = 'difficulty'
        self.timestamp = {'past': 0, 'now': 0, 'timer': TIMER *
DIFFICULTY_LEVELS[CURRENT_DIFFICULTY]['timer_modifier']}
        self.timer_up = False
        self.level =
DIFFICULTY_LEVELS[CURRENT_DIFFICULTY]['initial_level']
        self.tile_size = 100
        self.new_pattern = []
        self.input_pattern = []
        self.score = 0
        self.high_scores = {'easy': 0, 'medium': 0, 'hard': 0}
        self.difficulty = CURRENT_DIFFICULTY
        self.running = True
        self.waiting_input = False
        self.tile_animations = {}
        self.bg_offset = 0
        self.last_click = 0
        self.fade_alpha = 0
        self.pattern_index = 0

        # Timing variables
        self.tile_show_interval = 0.8  # Balanced for slower tile
display
        self.memorize_time = SHOWTIME * 0.75 / 1000
        self.next_tile_time = 0
        self.pattern_show_end_time = 0
        self.showing_pattern = False
        self.level_transition_time = 0
        self.show_level_message = False

        # Buttons
        self.buttons = {
            'easy': pygame.Rect(WIDTH//2 - 120, 180, 240, 60),
            'medium': pygame.Rect(WIDTH//2 - 120, 260, 240, 60),
            'hard': pygame.Rect(WIDTH//2 - 120, 340, 240, 60),
            'start': pygame.Rect(WIDTH//2 - 120, 420, 240, 60),
            'restart': pygame.Rect(WIDTH//2 - 150, 320, 300, 70),
            'leaderboard': pygame.Rect(WIDTH//2 - 150, 400, 300, 70),
            'quit': pygame.Rect(WIDTH//2 - 120, HEIGHT//2 + 140, 240,
60),
            'back_to_menu': pygame.Rect(WIDTH//2 - 150, HEIGHT//2, 300,
70)
        }

        # Screen instances
        self.screens = {
```

```
            'difficulty': DifficultyScreen(self),
            'playing': PlayingScreen(self),
            'gameover': GameOverScreen(self),
            'leaderboard': LeaderboardScreen(self),
            'paused': PauseScreen(self)
        }
        self.current_screen = self.screens[self.state]

    def getLeftTop(self, row, col):
        board_size = self.tile_size * self.level
        left = (self.tile_size * col) + (WIDTH - board_size) // 2
        top = (self.tile_size * row) + (HEIGHT - board_size) // 2
        return left, top

    def getClickedTile(self, x, y):
        for row in range(self.level):
            for col in range(self.level):
                left, top = self.getLeftTop(row, col)
                area = pygame.Rect(left, top, self.tile_size,
self.tile_size)
                if area.collidepoint(x, y):
                    return row, col
        return None, None

    def drawText(self, msg, x, y, color, title=False, shadow=True):
        font = self.big_font if title else self.normal_font
        if shadow:
            shadow_text = font.render(msg, True, (0, 0, 0))
            shadow_rect = shadow_text.get_rect(center=(x + 2, y + 2))
            self.win.blit(shadow_text, shadow_rect)
        text = font.render(msg, True, color)
        rect = text.get_rect(center=(x, y))
        self.win.blit(text, rect)

    def drawButton(self, text, rect, hover=False):
        color = BUTTON_HOVER_COLOR if hover else BUTTON_COLOR
        surface = pygame.Surface((rect.width, rect.height),
pygame.SRCALPHA)
        for i in range(rect.height):
            alpha = int(255 * (1 - i / rect.height) ** 2)
            pygame.draw.line(surface, (*color[:3], alpha), (0, i),
(rect.width, i))
        self.win.blit(surface, rect.topleft)
        pygame.draw.rect(self.win, NEON_GREEN, rect, 4,
border_radius=15)
        if hover:
            pygame.draw.rect(self.win, (0, 200, 0), rect, 2,
border_radius=15)
            rect = pygame.Rect(rect.left - 5, rect.top - 5, rect.width
* 1.05, rect.height * 1.05)
        text_surf = self.button_font.render(text, True, WHITE)
        text_rect = text_surf.get_rect(center=rect.center)
        self.win.blit(text_surf, text_rect)

    def drawAnimatedBackground(self):
        self.bg_offset = (self.bg_offset + 1) % HEIGHT
        for y in range(HEIGHT):
```

```
            ratio = ((y + self.bg_offset) % HEIGHT) / HEIGHT
            color = (
                int(BG_COLOR1[0] * (1 - ratio) + BG_COLOR2[0] * ratio),
                int(BG_COLOR1[1] * (1 - ratio) + BG_COLOR2[1] * ratio),
                int(BG_COLOR1[2] * (1 - ratio) + BG_COLOR2[2] * ratio)
            )
            pygame.draw.line(self.win, color, (0, y), (WIDTH, y))

    def lightUp(self, row, col, color):
        left, top = self.getLeftTop(row, col)
        surface = pygame.Surface((self.tile_size - 6, self.tile_size -
6), pygame.SRCALPHA)
        pygame.draw.rect(surface, color, (0, 0, self.tile_size - 6,
self.tile_size - 6), border_radius=15)
        self.win.blit(surface, (left, top))
        self.tile_animations[(row, col)] = {'scale': 1.0, 'alpha': 255,
'color': color}

    def runTimer(self):
        if self.timestamp['past']:
            self.timestamp['now'] = time.time()
            self.timestamp['timer'] -= (self.timestamp['now'] -
self.timestamp['past'])
            self.timestamp['past'] = self.timestamp['now']
            if self.timestamp['timer'] <= 0:
                self.timer_up = True
                self.timestamp['past'] = 0

    def resetGame(self):
        self.timer_up = False
        self.waiting_input = False
        self.score = 0
        self.level =
DIFFICULTY_LEVELS[self.difficulty]['initial_level']
        self.tile_size = 100
        self.timestamp = {'past': 0, 'now': 0, 'timer': TIMER *
DIFFICULTY_LEVELS[self.difficulty]['timer_modifier']}
        self.new_pattern = []
        self.input_pattern = []
        self.tile_animations = {}
        self.fade_alpha = 0
        self.pattern_index = 0
        self.next_tile_time = 0
        self.pattern_show_end_time = 0
        self.showing_pattern = False
        self.show_level_message = False
        self.level_transition_time = 0

    async def run(self):
        last_frame_time = time.time()
        while self.running:
            current_time = time.time()
            frame_time = current_time - last_frame_time
            last_frame_time = current_time

            for event in pygame.event.get():
                if event.type == pygame.QUIT:
```

```
                    self.running = False
                else:
                    await self.current_screen.handle_event(event)

            await self.current_screen.update()
            self.current_screen.draw()
            self.current_screen = self.screens.get(self.state,
self.screens['difficulty'])
            pygame.display.flip()
            self.clock.tick(FPS)
            if platform.system() == "Emscripten":
                await asyncio.sleep(1.0 / FPS)

        pygame.quit()
```

4. highscores.py

```
import json
import os

HIGH_SCORE_FILE = 'highscores.json'

def load_high_scores():
    if os.path.exists(HIGH_SCORE_FILE):
        try:
            with open(HIGH_SCORE_FILE, 'r') as f:
                return json.load(f)
        except json.JSONDecodeError:
            return {}
    return {}

def save_high_scores(high_scores):
    with open(HIGH_SCORE_FILE, 'w') as f:
        json.dump(high_scores, f, indent=4)
```

5. screens.py

```
6. import pygame
   import asyncio
   import random
   import time
   import math
   from settings import (
       WIDTH, HEIGHT, SHOWTIME, SHRINKAGE, TIMER, NEON_RED, NEON_GREEN,
   NEON_YELLOW,
       DARK_BG, GREY, DIFFICULTY_LEVELS, WHITE
   )

   class Screen:
       def __init__(self, game):
           self.game = game
```

```
    def draw(self):
        pass

    async def handle_event(self, event):
        pass

    async def update(self):
        pass

class DifficultyScreen(Screen):
    def draw(self):
        self.game.drawAnimatedBackground()
        self.game.drawText('Memtrix', WIDTH//2, 50, NEON_GREEN, True)
        self.game.drawText("Choose Difficulty", WIDTH//2, 120, WHITE)
        mouse_pos = pygame.mouse.get_pos()
        for diff in ['easy', 'medium', 'hard']:
            hover = self.game.buttons[diff].collidepoint(mouse_pos)
            self.game.drawButton(diff.capitalize(),
self.game.buttons[diff], hover)

    async def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONUP:
            current_time = time.time()
            if (current_time - self.game.last_click) > 0.2:
                self.game.last_click = current_time
                for diff in ['easy', 'medium', 'hard']:
                    if self.game.buttons[diff].collidepoint(event.pos):
                        self.game.difficulty = diff
                        self.game.state = 'playing'
                        self.game.level =
DIFFICULTY_LEVELS[self.game.difficulty]['initial_level']
                        self.game.timestamp['past'] = time.time()
                        self.game.resetGame()

class PlayingScreen(Screen):
    def __init__(self, game):
        super().__init__(game)
        self.pattern_start_time = 0

    def draw(self):
        self.game.drawAnimatedBackground()
        self.game.drawText(f'Score: {self.game.score}', 30, 20,
NEON_GREEN)
        self.game.drawText(f'Level: {self.game.level - 1}', WIDTH//2 -
30, 20, NEON_GREEN)
        self.game.drawText(f'Best:
{self.game.high_scores.get(self.game.difficulty, 0)}', WIDTH - 130, 20,
NEON_GREEN)
        board_size = self.game.tile_size * self.game.level
        left, top = self.game.getLeftTop(0, 0)
        for row in range(self.game.level):
            for col in range(self.game.level):
                tile_left, tile_top = self.game.getLeftTop(row, col)
                size = self.game.tile_size - 6
                anim_key = (row, col)
                if anim_key in self.game.tile_animations:
                    scale =
```

```
self.game.tile_animations[anim_key]['scale']
                    alpha =
self.game.tile_animations[anim_key]['alpha']
                    color =
self.game.tile_animations[anim_key]['color']
                    size *= 1 + 0.1 * (1 - scale)
                    offset = (self.game.tile_size - size) / 2
                    surface = pygame.Surface((size, size),
pygame.SRCALPHA)
                    pygame.draw.rect(surface, (*color[:3], int(alpha)),
(0, 0, size, size), border_radius=5)
                    self.game.win.blit(surface, (tile_left + offset,
tile_top + offset))
                    if self.game.difficulty != 'easy':
                        self.game.tile_animations[anim_key]['scale'] -=
0.05
                        self.game.tile_animations[anim_key]['alpha'] -=
15
                        if self.game.tile_animations[anim_key]['scale']
<= 0 or self.game.tile_animations[anim_key]['alpha'] <= 0:
                            del self.game.tile_animations[anim_key]
                else:
                    surface = pygame.Surface((size, size),
pygame.SRCALPHA)
                    color = GREY
                    pygame.draw.rect(surface, (*color[:3], 50), (0, 0,
size, size), border_radius=5)
                    pygame.draw.rect(surface, color, (0, 0, size,
size), border_radius=2)
                    self.game.win.blit(surface, (tile_left, tile_top))
        border_width = 4 + math.sin(time.time())
        pygame.draw.rect(self.game.win, NEON_RED, (left - 5, top - 5,
board_size + 10, board_size + 10), int(border_width // 2),
border_radius=5)
        time_ratio = max(0, self.game.timestamp['timer'] / (TIMER *
DIFFICULTY_LEVELS[self.game.difficulty]['timer_modifier']))
        timer_width = int((WIDTH - 40) * time_ratio)
        pygame.draw.rect(self.game.win, DARK_BG, (20, HEIGHT - 50,
WIDTH - 40, 10), border_radius=5)
        pygame.draw.rect(self.game.win, NEON_GREEN, (20, HEIGHT - 50,
timer_width, 10), border_radius=5)
        pygame.draw.rect(self.game.win, NEON_GREEN, (20, HEIGHT - 50,
WIDTH - 40, 10), 2, border_radius=5)
        if self.game.showing_pattern:
            self.game.drawText('Memorize this pattern.', WIDTH//2,
HEIGHT - 70, WHITE)
        if self.game.show_level_message:
            if self.game.level <=
DIFFICULTY_LEVELS[self.game.difficulty]['max_level']:
                self.game.drawText('Level Up!', WIDTH//2, HEIGHT//2,
NEON_GREEN, True)
            else:
                self.game.drawText('Max Level! Keep Going!', WIDTH//2,
HEIGHT//2, NEON_GREEN, True)
            if time.time() >= self.game.level_transition_time:
                self.game.show_level_message = False
```

```
    async def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONUP and
self.game.waiting_input and not self.game.timer_up:
            current_time = time.time()
            if current_time - self.game.last_click >= 0.2:
                self.game.last_click = current_time
                row, col = self.game.getClickedTile(*event.pos)
                if row is not None and (row, col) not in
self.game.input_pattern:
                    self.game.input_pattern.append((row, col))
                    self.game.tile_animations.clear()
                    for (r, c) in self.game.input_pattern:
                        color = NEON_GREEN if (r, c) in
self.game.new_pattern else NEON_RED
                        self.game.lightUp(r, c, color)
                    if len(self.game.input_pattern) ==
len(self.game.new_pattern):
                        self.game.waiting_input = False
                        self.game.tile_animations.clear()
                        if self.game.input_pattern ==
self.game.new_pattern:
                            self.game.score += 10
                            self.game.high_scores[self.game.difficulty]
= max(self.game.score, self.game.high_scores.get(self.game.difficulty,
0))
                            self.game.timestamp['timer'] = TIMER *
DIFFICULTY_LEVELS[self.game.difficulty]['timer_modifier']
                            self.game.timestamp['past'] = current_time
                            for (r, c) in self.game.new_pattern:
                                self.game.lightUp(r, c, NEON_GREEN)
                            self.game.show_level_message = True
                            self.game.level_transition_time =
current_time + 1.0
                            self.game.level += 1
                            self.game.tile_size = max(50,
self.game.tile_size - 10)
                            self.game.new_pattern = []
                            self.game.input_pattern = []
                            self.game.pattern_index = 0
                            self.game.showing_pattern = True
                            self.game.next_tile_time = current_time +
self.game.tile_show_interval
                            self.game.pattern_show_end_time = 0
                            self.game.new_pattern =
self.generatePattern()
                            self.pattern_start_time = current_time
                            self.game.tile_animations.clear()
                            self.game.timestamp['past'] = current_time
                        else:
                            self.game.high_scores[self.game.difficulty]
= max(self.game.score, self.game.high_scores.get(self.game.difficulty,
0))
                            self.game.state = 'gameover'
        elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_ESCAPE:
            self.game.state = 'paused'
```

```
    async def update(self):
        self.game.runTimer()
        if self.game.timer_up:
            self.game.high_scores[self.game.difficulty] =
max(self.game.score, self.game.high_scores.get(self.game.difficulty,
0))
            self.game.state = 'gameover'
            self.game.waiting_input = False
            self.game.tile_animations.clear()
        elif not self.game.waiting_input and not self.game.timer_up:
            current_time = time.time()
            tile_show_duration = 1.0 if self.game.difficulty == 'easy'
else self.game.tile_show_interval
            if not self.game.new_pattern or len(self.game.new_pattern)
!= self.game.level:
                self.game.new_pattern = self.generatePattern()
                self.game.pattern_index = 0
                self.game.showing_pattern = True
                self.game.next_tile_time = current_time +
tile_show_duration
                self.game.pattern_show_end_time = 0
                self.pattern_start_time = current_time
            elif self.game.showing_pattern:
                if current_time - self.pattern_start_time > 5:
                    self.game.showing_pattern = False
                    self.game.waiting_input = True
                    self.game.tile_animations.clear()
                    self.game.pattern_index = 0
                elif self.game.pattern_index >=
len(self.game.new_pattern):
                    if current_time >= self.game.pattern_show_end_time:
                        self.game.showing_pattern = False
                        self.game.waiting_input = True
                        self.game.timestamp['past'] = current_time
                        self.game.tile_animations.clear()
                        self.game.pattern_index = 0
                elif current_time >= self.game.next_tile_time:
                    row, col =
self.game.new_pattern[self.game.pattern_index]
                    if (row, col) not in self.game.tile_animations:
                        self.game.lightUp(row, col, NEON_GREEN)
                        self.game.tile_animations[(row, col)] =
{'scale': 1.0, 'alpha': 255, 'color': NEON_GREEN}
                    self.game.pattern_index += 1
                    if self.game.pattern_index <
len(self.game.new_pattern):
                        self.game.next_tile_time = current_time +
tile_show_duration
                    else:
                        self.game.pattern_show_end_time = current_time
+ self.game.memorize_time
                        if self.game.difficulty == 'easy':
                            self.game.pattern_show_end_time +=
tile_show_duration

    def generatePattern(self):
        pattern = []
```

```
        while len(pattern) < self.game.level:
            row = random.randrange(self.game.level)
            col = random.randrange(self.game.level)
            if (row, col) not in pattern:
                pattern.append((row, col))
        return pattern

class GameOverScreen(Screen):
    def draw(self):
        self.game.drawAnimatedBackground()
        self.game.drawText('Game Over!', WIDTH//2, 100, NEON_RED, True)
        self.game.drawText(f'Score: {self.game.score}', WIDTH//2, 180,
WHITE, shadow=True)
        self.game.drawText(f'Best:
{self.game.high_scores.get(self.game.difficulty, 0)}', WIDTH//2, 220,
WHITE, shadow=True)
        if self.game.score > 0:
            self.game.drawText('Good Effort!', WIDTH//2, 260,
NEON_GREEN, shadow=True)
        mouse_pos = pygame.mouse.get_pos()
        hover_restart =
self.game.buttons['restart'].collidepoint(mouse_pos)
        hover_leaderboard =
self.game.buttons['leaderboard'].collidepoint(mouse_pos)
        self.game.drawButton('Play Again',
self.game.buttons['restart'], hover_restart)
        self.game.drawButton('Leaderboard',
self.game.buttons['leaderboard'], hover_leaderboard)

    async def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONUP:
            current_time = time.time()
            if (current_time - self.game.last_click) > 0.2:
                self.game.last_click = current_time
                if
self.game.buttons['leaderboard'].collidepoint(event.pos):
                    self.game.state = 'leaderboard'
                elif
self.game.buttons['restart'].collidepoint(event.pos):
                    self.game.resetGame()
                    self.game.state = 'playing'

class LeaderboardScreen(Screen):
    def draw(self):
        self.game.drawAnimatedBackground()
        self.game.drawText('Leaderboard', WIDTH//2, 50, NEON_GREEN,
True)
        y = 150
        for diff, score in sorted(self.game.high_scores.items(),
key=lambda x: x[1], reverse=True):
            self.game.drawText(f'{diff.capitalize()}: {score}',
WIDTH//2, y, WHITE)
            y += 50
        mouse_pos = pygame.mouse.get_pos()
        hover = self.game.buttons['restart'].collidepoint(mouse_pos)
        self.game.drawButton('Play Again',
self.game.buttons['restart'], hover)
```

```
        async def handle_event(self, event):
            if event.type == pygame.MOUSEBUTTONUP:
                current_time = time.time()
                if (current_time - self.game.last_click) > 0.2:
                    self.game.last_click = current_time
                    if
    self.game.buttons['restart'].collidepoint(event.pos):
                        self.game.resetGame()
                        self.game.state = 'playing'

    class PauseScreen(Screen):
        def draw(self):
            self.game.drawAnimatedBackground()
            self.game.drawText('Paused', WIDTH//2, HEIGHT//4, NEON_YELLOW,
    True)
            mouse_pos = pygame.mouse.get_pos()
            hover_back =
    self.game.buttons['back_to_menu'].collidepoint(mouse_pos)
            self.game.drawButton('Back to Menu',
    self.game.buttons['back_to_menu'], hover_back)

        async def handle_event(self, event):
            if event.type == pygame.MOUSEBUTTONUP:
                current_time = time.time()
                if (current_time - self.game.last_click) > 0.2:
                    self.game.last_click = current_time
                    if
    self.game.buttons['back_to_menu'].collidepoint(event.pos):
                        self.game.state = 'difficulty'
                        self.game.resetGame()
            elif event.type == pygame.KEYDOWN and event.key ==
    pygame.K_ESCAPE:
                self.game.state = 'playing'
```

5.Settings.py

```
import pygame

# Screen Constants
WIDTH = 600
HEIGHT = 600

# Font and Gameplay constants
FONTSIZE = 30
SHOWTIME = 1000
SHRINKAGE = 9
TIMER = 60  # Base timer, adjusted by difficulty
FPS = 60  # Increased for smoother animations

# Difficulty settings
```

```
DIFFICULTY_LEVELS = {
    'easy': {'initial_level': 2, 'timer_modifier': 1.5, 'max_level': 4},
    'medium': {'initial_level': 4, 'timer_modifier': 1.0, 'max_level': 8},
    'hard': {'initial_level': 5, 'timer_modifier': 0.8, 'max_level': 10}
}
CURRENT_DIFFICULTY = 'medium'  # Default difficulty

# Colors
WHITE = (255, 255, 255)
NEON_RED = (255, 50, 50)
NEON_YELLOW = (255, 255, 50)  # Ensure this line is present
DARK_BG = (15, 15, 30)
GREY = (100, 120, 140)
NEON_GREEN = (50, 255, 50)
BG_COLOR1 = (20, 30, 50)  # Gradient start
BG_COLOR2 = (10, 50, 70)  # Gradient end
BUTTON_COLOR = (50, 70, 90)
BUTTON_HOVER_COLOR = (70, 90, 110)
SHADOW_COLOR = (10, 10, 10)

# Initialize pygame fonts
pygame.font.init()
```

Conclusion

The **Memtrix** project demonstrates effective application of object-oriented design in game development. With responsive design, async compatibility, and persistent scores, it provides a complete user experience. The modular architecture ensures maintainability and expandability, serving as a great showcase for both technical skill and creative UI design.